

Project Information Migration Analyze

English technical specification and read-only Groovy analyze script

Prepared for Jira Data Center / ScriptRunner migration validation. Generated on 18 August 2026.

Purpose

This document describes and contains the final read-only analyze script for migrating Project Information data into the new Project Information n architecture. The script creates a CSV report only. It does not update Jira issues, create options, enable options, disable options, or dispatch events.

The analyze scope is all Jira issues where SAP-Order-No is not empty. The migration truth is the existing old Project Information value, and the report explicitly marks conflicts when already populated new fields differ from the simulated migration result.

Field mapping

Field	ID	Role in migration
Project Information n	18702	New select-list carrier of the functional value
Project Information	15600	Old source field and primary migration truth
Project Information n	20200	Text mirror for JQL search, same text value as 18702
Project Information Override Key	20201	Authority key used by listener and consistency job
SAP-Order-No	10217	Analyze scope filter and fallback value source
SAP-Order-No input	10225	Fallback value source if old Project Information is empty
Parent Link	10301	Hierarchy parent resolution
Epic Link	10001	Hierarchy parent resolution

Migration rules

- Every non-empty value from old Project Information 15600 is preserved as migration truth.
- If the old value is not available as an option in Project Information n 18702, the CSV marks it as a legacy option requirement. This is not treated as a migration failure.
- Variant A is used: when a child has an old value different from the parent authority, the child becomes an override root or nested override root. This is not treated as a conflict.
- Current values in 18702, 20200, and 20201 are compared against the simulated result. If a current value is non-empty and different, the row is marked as conflict for manual review.

Conflict classification

A conflict exists only when an already populated new field differs from the simulated migration value. Empty current values are a normal would-change case, not a conflict.

Conflict type	Meaning
NO_CONFLICT	No populated new field differs from the simulated value
CONFLICT_PI_VALUE	Current 18702 value differs from simulated 18702 value
CONFLICT_TEXT_MIRROR	Current 20200 value differs from simulated 20200 value
CONFLICT_OVERRIDE_KEY	Current 20201 value differs from simulated override key
CONFLICT_ALL	All three new fields differ from the simulated values
CONFLICT_REQUIRES_LEGACY_OPTION	Conflict exists and the simulated value also needs a legacy option

Conflict type	Meaning
CONFLICT_REQUIRES_DISABLED_OPTION_HANDLING	Conflict exists and the option is currently disabled

Recommended CSV review order

- Filter result values starting with CONFLICT.
- Filter legacyOptionAction not equal to NONE.
- Review branch values OVERRIDE_ROOT, NESTED_OVERRIDE_ROOT, and ROOT_OVERRIDE_OR_STANDALONE_AUTHORITY.
- Review unresolved and cycle-detected rows before any write migration.

Appendix A - Full Groovy analyze script

The code below is the read-only ScriptRunner analyze job. It generates the CSV report and performs no write operation.

```
package jobs.global.project_information_change_old.migration

import com.atlassian.jira.bc.issue.search.SearchService
import com.atlassian.jira.component.ComponentAccessor
import com.atlassian.jira.issue.Issue
import com.atlassian.jira.issue.customfields.option.Option
import com.atlassian.jira.issue.fields.config.FieldConfig
import com.atlassian.jira.user.ApplicationUser
import com.atlassian.jira.web.bean.PagerFilter
import groovy.json.JsonSlurper
import org.apache.log4j.Logger

import java.nio.charset.StandardCharsets
import java.text.SimpleDateFormat

/**
 * ProjectInfoMigrationAnalyze
 *
 * READ-ONLY simulation for Project Information migration.
 *
 * Scope:
 *   All issues where SAP-Order-No is not empty:
 *   cf[10217] is not EMPTY
 *
 * Simulates:
 *   - old Project Information 15600 to Project Information n 18702
 *   - text mirror 20200 synchronization
 *   - Project Information Override Key 20201
 *   - inheritance based on nearest authority issue
 *   - legacy option requirements per field context
 *   - conflicts between already existing new-field values and simulated migration values
 *
 * Does not:
 *   - update issues
 *   - create options
 *   - enable options
 *   - disable options
 *   - dispatch events
 *
 * Main migration rule:
 *   Every non-empty value from old Project Information 15600 is migration truth.
 *   It is preserved exactly. If it does not exist as an option in 18702, the CSV
 *   reports that a legacy option must be created for the relevant field context.
 *
 * Conflict rule:
 *   If current new fields already contain a non-empty value that differs from the
 *   simulated migration result, the row is marked as conflict.
 *
 *   This does not apply to old child value versus parent inherited value.
 *   Variant A is used:
 *   Child old value different from parent authority means OVERRIDE_ROOT or
 *   NESTED_OVERRIDE_ROOT, not conflict.
 *
 * Authority model:
 *   - Initiative with value:
 *     source authority, override key empty
 *   - Non-Initiative root with own value:
 *     standalone authority, override key = own key
 *   - Non-Initiative child with own value different from inherited value:
 *     override root, override key = own key
 *   - Issue inheriting from Initiative:
 *     override key empty
 *   - Issue inheriting from override root or standalone authority:
 *     override key = authority issue key
 *
 * Output:
 *   {jira.home}/export/project-info-analyze/project-info-analyze-{timestamp}.csv
 *
 * @author chabrecek.anton
 */
class ProjectInfoMigrationAnalyze {

    private static final int BATCH_SIZE = 200
    private static final String SEPARATOR = ","

    // New Project Information select list, carrier of functional value.
    private static final String CF_PROJECT_INFORMATION_N = "customfield_18702"

    // Old Project Information, migration source.
    private static final String CF_OLD_PROJECT_INFORMATION = "customfield_15600"

    // Text mirror for JQL.
    private static final String CF_PROJECT_INFORMATION_TEXT = "customfield_20200"

    // New override key.
    private static final String CF_PROJECT_INFORMATION_OVERRIDE_KEY = "customfield_20201"

    // Existing SAP fields.
    private static final String CF_SAP_INPUT = "customfield_10225"
    private static final String CF_SAP_SCRIPTED = "customfield_10217"

    // Hierarchy fields.
```

```

private static final String CF_PARENT_LINK = "customfield_10301"
private static final String CF_EPIC_LINK = "customfield_10001"

private static final String ISSUE_TYPE_INITIATIVE = "Initiative"

private static final String LABEL_MULTIPLE = "Multiple project with [%s]"
private static final String LABEL_HISTORY = "history [%s]"

private static final String JIRA_BOT = "jira.bot"

/**
 * Requested analyze scope.
 *
 * If your Jira instance resolves scripted field names more reliably than IDs,
 * you can replace this with:
 * \SAP-Order-No\ is not EMPTY order by key ASC
 */
private static final String JQL_SCOPE = "cf[10217] is not EMPTY order by key ASC"

private static final String[] CSV_HEADER = [
    "issueKey",
    "projectKey",
    "issueType",
    "summary",
    "parentKey",
    "parentInOutScope",

    "currentProjectInformationN",
    "simulatedProjectInformationN",
    "currentProjectInformationText",
    "simulatedProjectInformationText",
    "currentOverrideKey",
    "simulatedOverrideKey",

    "oldProjectInformationRaw",
    "oldProjectInformationExtracted",
    "oldProjectInformationSap",
    "sapInput",

    "sapScripted",

    "directResolutionSource",
    "optionContextId",
    "optionExistsInContext",
    "optionDisabledInContext",
    "optionExistsInAnyContext",
    "legacyOptionAction",
    "optionLifecycleAction",

    "branch",
    "sourceIssue",
    "authorityValue",

    "wouldChangeProjectInformationN",
    "wouldChangeTextMirror",
    "wouldChangeOverrideKey",

    "conflict",
    "conflictType",
    "conflictDetail",

    "result",
    "detail"
]

private final Logger log = Logger.getLogger("scriptrunner.analyze.project-info")

private final def cfManager = ComponentAccessor.getCustomFieldManager()
private final def issueManager = ComponentAccessor.getIssueManager()
private final def optManager = ComponentAccessor.getOptionsManager()
private final def userManager = ComponentAccessor.getUserManager()
private final def searchService = ComponentAccessor.getComponent(SearchService)
private final def jiraHomeComponent =
    ComponentAccessor.getComponent(com.atlassian.jira.config.util.JiraHome)

private final def cfNewPI = cfManager.getCustomFieldObject(CF_PROJECT_INFORMATION_N)
private final def cfOldPI = cfManager.getCustomFieldObject(CF_OLD_PROJECT_INFORMATION)
private final def cfTextMirror = cfManager.getCustomFieldObject(CF_PROJECT_INFORMATION_TEXT)
private final def cfOverrideKey = cfManager.getCustomFieldObject(CF_PROJECT_INFORMATION_OVERRIDE_KEY)
private final def cfSapInput = cfManager.getCustomFieldObject(CF_SAP_INPUT)
private final def cfSapScripted = cfManager.getCustomFieldObject(CF_SAP_SCRIPTED)
private final def cfParentLink = cfManager.getCustomFieldObject(CF_PARENT_LINK)
private final def cfEpicLink = cfManager.getCustomFieldObject(CF_EPIC_LINK)

// Option indexes.
private List<FieldConfig> fieldConfigs = []
private Map<Long, Map<String, Option>> optionsByConfig = [:]
private Map<String, List<Map>> optionOccurrencesByValue = [:]
private Map<String, List<Option>> sapToOptions = [:]

// Output scope.
private Map<Long, Issue> outputIssuesById = [:]
private Map<String, Issue> outputIssuesByKey = [:]

// Simulation caches. They can contain parent issues outside output scope.
private Map<Long, Map> directCache = [:]
private Map<Long, Map> simulationCache = [:]
private Map<Long, Issue> parentCache = [:]

```

```

private Set<Long> recursionStack = new HashSet<>()

// Counters for output scope only. Updated while writing CSV.
private int totalOutputIssues = 0
private int totalErrors = 0

private int totalOk = 0
private int totalWouldChange = 0
private int totalConflicts = 0
private int totalLegacyOptionsRequired = 0
private int totalDisabledOptionHandling = 0
private int totalUnresolved = 0
private int totalStandaloneAuthorities = 0
private int totalOverrideRoots = 0
private int totalNestedOverrideRoots = 0

String run() {
    log.info("=== PROJECT INFORMATION ANALYZE START ===")
    log.info("Scope JQL: ${JQL_SCOPE}")

    ApplicationUser botUser = userManager.getUserByName(JIRA_BOT)
    if (!botUser) {
        log.error("User '${JIRA_BOT}' not found.")
        return "FAILED: bot user not found"
    }

    String validationError = validateConfiguration()
    if (validationError) {
        log.error(validationError)
        return "FAILED: ${validationError}"
    }

    buildOptionIndexes()

    File outputFile = prepareOutputFile()
    if (!outputFile) {
        return "FAILED: could not create output file"
    }

    loadOutputIssues(botUser)

    log.info("-- Simulating scoped issues and required parent authority chain --")
    outputIssuesById.values().sort { it.key }.each { Issue issue ->
        try {
            simulateIssue(issue)
        } catch (Exception e) {
            totalErrors++
            log.error("Simulation failed for ${issue?.key}: ${e.message}", e)
            simulationCache[issue.id] = [
                simulatedPi: "",
                simulatedText: "",
                simulatedOverrideKey: "",
                branch: "ERROR",
                sourceIssue: "",
                authorityValue: "",
                directResolutionSource: "ERROR",
                result: "ERROR",
                detail: "${e.class.simpleName}: ${e.message}",
                optionInfo: emptyOptionInfo(issue)
            ]
        }
    }

    log.info("-- Writing CSV --")
    outputFile.withWriter(StandardCharsets.UTF_8.name()) { Writer writer ->
        writer.write("\uFEFF")
        writer.writeLine(CSV_HEADER.join(SEPARATOR))

        outputIssuesById.values().sort { it.key }.each { Issue issue ->
            writer.writeLine(buildCsvLineAndUpdateCounters(issue))
        }

        writer.flush()
    }

    log.info("=== PROJECT INFORMATION ANALYZE END ===")
    log.info(" Output issues: ${totalOutputIssues}")
    log.info(" OK: ${totalOk}")
    log.info(" Would change: ${totalWouldChange}")
    log.info(" Conflicts: ${totalConflicts}")
    log.info(" Legacy options required: ${totalLegacyOptionsRequired}")
    log.info(" Disabled option handling: ${totalDisabledOptionHandling}")
    log.info(" Unresolved: ${totalUnresolved}")
    log.info(" Standalone authorities: ${totalStandaloneAuthorities}")
    log.info(" Override roots: ${totalOverrideRoots}")
    log.info(" Nested override roots: ${totalNestedOverrideRoots}")
    log.info(" Errors: ${totalErrors}")
    log.info(" Output: ${outputFile.absolutePath}")

    return "Done. ${totalOutputIssues} issues analyzed, ${totalConflicts} conflicts, ${totalErrors}
        errors. File: ${outputFile.absolutePath}"
}

private String validateConfiguration() {
    List<String> missing = []

    if (!cfNewPI) missing.add("${CF_PROJECT_INFORMATION_N} Project Information n select")
    if (!cfOldPI) missing.add("${CF_OLD_PROJECT_INFORMATION} old Project Information")
    if (!cfTextMirror) missing.add("${CF_PROJECT_INFORMATION_TEXT} Project Information n text")
}

```

```

        mirror")
    if (!cfOverrideKey) missing.add("${CF_PROJECT_INFORMATION_OVERRIDE_KEY} Project Information
        Override Key")
    if (!cfSapScripted) missing.add("${CF_SAP_SCRIPTED} SAP-Order-No")
    if (!cfSapInput) missing.add("${CF_SAP_INPUT} SAP-Order-No input")
    if (!cfParentLink) missing.add("${CF_PARENT_LINK} Parent Link")
    if (!cfEpicLink) missing.add("${CF_EPIC_LINK} Epic Link")

    if (!missing.isEmpty()) {
        return "Missing custom field(s): ${missing.join(', ')}"
    }

    return null
}

private void buildOptionIndexes() {
    log.info("-- Building Project Information n option indexes --")

    fieldConfigs = (cfNewPI.configurationSchemes
        ?.collectMany { it.configs?.values() ?: [] }
        ?.unique { it.id }) ?: []

    fieldConfigs.each { FieldConfig config ->
        Long configId = config.id as Long
        optionsByConfig.computeIfAbsent(configId) { [:] }

        optManager.getOptions(config)?.each { Option opt ->
            String value = opt?.value?.toString()?.trim()
            if (!value) {
                return
            }

            String norm = normalize(value)
            optionsByConfig[configId][norm] = opt

            optionOccurrencesByValue
                .computeIfAbsent(norm) { [] }
                .add([
                    configId: configId,
                    optionId: opt.optionId,
                    disabled: isDisabledOption(opt),
                    value: value
                ])

            String sap = extractSap(value)
            if (sap) {
                sapToOptions.computeIfAbsent(sap) { [] }.add(opt)
            }
        }
    }

    int optionCount = optionOccurrencesByValue.values().sum { it.size() } ?: 0
    int duplicateValueCount = optionOccurrencesByValue.count { k, v -> v.size() > 1 }
    int ambiguousSapCount = sapToOptions.count { k, v -> v.size() > 1 }

    log.info(" Field configs: ${fieldConfigs.size()}")
    log.info(" Option occurrences: ${optionCount}")
    log.info(" Values present in multiple contexts/options: ${duplicateValueCount}")
    log.info(" SAP numbers indexed: ${sapToOptions.size()}")
    log.info(" Ambiguous SAP numbers: ${ambiguousSapCount}")
}

private void loadOutputIssues(ApplicationUser user) {
    log.info("-- Loading output issues --")

    def parseResult = searchService.parseQuery(user, JQL_SCOPE)
    if (!parseResult.valid) {
        throw new IllegalArgumentException("Invalid JQL: ${JQL_SCOPE}. Errors:
            ${parseResult.errors}")
    }

    int start = 0
    int total = 0

    while (true) {
        def pager = new PagerFilter(BATCH_SIZE)
        pager.start = start

        def searchResult = searchService.search(user, parseResult.query, pager)
        def hits = searchResult.results

        if (!hits) {
            break
        }

        hits.each { hit ->
            try {
                Issue issue = issueManager.getIssueObject(hit.id)
                if (issue) {
                    outputIssuesById[issue.id] = issue
                    outputIssuesByKey[issue.key] = issue
                }
            } catch (Exception e) {
                totalErrors++
                log.error("Failed to load issue ${hit?.key ?: hit?.id}: ${e.message}", e)
            }
        }
    }
}

```

```

        total += hits.size()
        log.info("  Batch: ${hits.size()}, total loaded: ${total}")

        start += hits.size()
        if (hits.size() < BATCH_SIZE) {
            break
        }
    }

    totalOutputIssues = outputIssuesById.size()
    log.info("Loaded output issues: ${totalOutputIssues}")
}

/**
 * Recursive, parent-first simulation.
 *
 * Every output issue is simulated. Parent issues outside the output scope may
 * be loaded and simulated only to determine inherited authority.
 */
private Map simulateIssue(Issue issue) {
    if (!issue) {
        return unresolvedSynthetic("UNKNOWN", "NO_ISSUE", "Issue object is null.")
    }

    if (simulationCache.containsKey(issue.id)) {
        return simulationCache[issue.id]
    }

    if (recursionStack.contains(issue.id)) {
        Map cycle = buildUnresolvedSimulation(
            issue,
            "CYCLE_DETECTED",
            "Cycle detected while walking parent hierarchy."
        )
        simulationCache[issue.id] = cycle
        return cycle
    }

    recursionStack.add(issue.id)

    Map direct = getDirectResolution(issue)
    Issue parent = getParent(issue)
    Map parentSim = parent ? simulateIssue(parent) : null

    Map sim

    if (isInitiative(issue)) {
        if (direct.value) {
            sim = buildSimulation(
                issue,
                direct.value as String,
                "",
                "INITIATIVE",
                issue.key,
                direct.value as String,
                direct,
                ""
            )
        } else {
            sim = buildUnresolvedSimulation(
                issue,
                "UNRESOLVED_INITIATIVE_VALUE",
                "Initiative has no old Project Information value and no SAP fallback value."
            )
        }

        simulationCache[issue.id] = sim
        recursionStack.remove(issue.id)
        return sim
    }

    boolean parentHasAuthority = parentSim && parentSim.simulatedPi
    String inheritedValue = parentHasAuthority ? parentSim.simulatedPi?.toString() : null
    String inheritedSourceIssue = parentHasAuthority ? parentSim.sourceIssue?.toString() : null
    String inheritedOverrideKey = parentHasAuthority ? parentSim.simulatedOverrideKey?.toString() :
    ""

    if (!parentHasAuthority) {
        if (direct.value) {
            sim = buildSimulation(
                issue,
                direct.value as String,
                issue.key,
                "ROOT_OVERRIDE_OR_STANDALONE_AUTHORITY",
                issue.key,
                direct.value as String,
                direct,
                ""
            )
        } else {
            sim = buildUnresolvedSimulation(
                issue,
                "UNRESOLVED_NO_OLD_VALUE_NO_AUTHORITY",
                "Issue has no old Project Information value, no SAP fallback value and no parent
                authority."
            )
        }
    }
}

```

```

        simulationCache[issue.id] = sim
        recursionStack.remove(issue.id)
        return sim
    }

    /**
     * Variant A:
     * If the issue has its own old value and it differs from inherited authority,
     * it becomes an override root. This is not a conflict.
     */
    if (direct.value && normalize(direct.value as String) != normalize(inheritedValue)) {
        String branch = inheritedOverrideKey ? "NESTED_OVERRIDE_ROOT" : "OVERRIDE_ROOT"

        sim = buildSimulation(
            issue,
            direct.value as String,
            issue.key,
            branch,
            issue.key,
            direct.value as String,
            direct,
            "Own old Project Information differs from parent authority. Variant A applies: issue becomes override root."
        )

        simulationCache[issue.id] = sim
        recursionStack.remove(issue.id)
        return sim
    }

    String branch
    if (inheritedOverrideKey) {
        branch = direct.value ? "INHERITED_FROM_OVERRIDE_SAME_VALUE" : "INHERITED_FROM_OVERRIDE"
    } else {
        branch = direct.value ? "INHERITED_FROM_INITIATIVE_SAME_VALUE" : "INHERITED_FROM_INITIATIVE"
    }

    sim = buildSimulation(
        issue,
        inheritedValue,
        inheritedOverrideKey ? "" : "",
        branch,
        inheritedSourceIssue ? "" : "",
        inheritedValue,
        direct.value ? direct : inheritedDirectInfo(),
        ""
    )

    simulationCache[issue.id] = sim
    recursionStack.remove(issue.id)
    return sim
}

/**
 * Direct value priority:
 * 1. Old Project Information 15600 extracted value, preserved exactly.
 * 2. SAP-Order-No input, only if old value is missing.
 * 3. SAP-Order-No scripted, only if old value and SAP input are missing.
 * 4. No direct value.
 *
 * Current values in 18702, 20200 and 20201 are not authority.
 * They are compared later and reported as conflicts if they differ.
 */
private Map getDirectResolution(Issue issue) {
    if (directCache.containsKey(issue.id)) {
        return directCache[issue.id]
    }

    Object oldObject = issue.getCustomFieldValue(cfOldPI)
    String oldRaw = oldObject?.toString()
    String oldExtracted = extractDbRow(oldObject)?.trim()
    String oldSap = extractSap(oldExtracted)
    String sapInput = issue.getCustomFieldValue(cfSapInput)?.toString()?.trim()
    String sapScripted = issue.getCustomFieldValue(cfSapScripted)?.toString()?.trim()

    Map direct

    if (oldExtracted) {
        Map optInfo = inspectOptionAvailability(issue, oldExtracted)
        direct = [
            value: oldExtracted,
            source: optInfo.legacyOptionAction == "NONE" ?
                "OLD_PI_EXISTING_OPTION" :
                (optInfo.legacyOptionAction == "TEMPORARILY_ENABLE_DISABLED_OPTION" ?
                    "OLD_PI_DISABLED_OPTION" :
                    "OLD_PI_LEGACY_OPTION_REQUIRED"),
            oldRaw: oldRaw ? "" : "",
            oldExtracted: oldExtracted,
            oldSap: oldSap ? "" : "",
            sapInput: sapInput ? "" : "",
            sapScripted: sapScripted ? "" : "",
            optionInfo: optInfo,
            detail: optInfo.legacyOptionAction == "NONE" ?
                "Old Project Information value exists as option in this issue context." :
                "Old Project Information value is preserved and requires option preparation for this issue context."
        ]
    }

    directCache[issue.id] = direct
}

```



```

        return direct
    }

    if (sapInput) {
        direct = resolveBySapForAnalyze(issue, sapInput, "SAP_INPUT")
        direct.oldRaw = oldRaw ?: ""
        direct.oldExtracted = ""
        direct.oldSap = ""
        direct.sapInput = sapInput ?: ""
        direct.sapScripted = sapScripted ?: ""
        directCache[issue.id] = direct
        return direct
    }

    if (sapScripted && sapScripted != "NOT Defined") {
        direct = resolveBySapForAnalyze(issue, sapScripted, "SAP_SCRIPTED")
        direct.oldRaw = oldRaw ?: ""
        direct.oldExtracted = ""
        direct.oldSap = ""
        direct.sapInput = sapInput ?: ""
        direct.sapScripted = sapScripted ?: ""
        directCache[issue.id] = direct
        return direct
    }
}

direct = [
    value: "",
    source: "NO_DIRECT_VALUE",
    oldRaw: oldRaw ?: "",
    oldExtracted: "",
    oldSap: "",
    sapInput: sapInput ?: "",
    sapScripted: sapScripted ?: "",
    optionInfo: emptyOptionInfo(issue),
    detail: "No old Project Information value and no usable SAP fallback value."
]

directCache[issue.id] = direct
return direct
}

private Map resolveBySapForAnalyze(Issue issue, String rawSap, String typePrefix) {
    String sap = rawSap?.replaceAll("[^0-9]", "")
    if (!sap) {
        return [
            value: "",
            source: "${typePrefix}_INVALID",
            optionInfo: emptyOptionInfo(issue),
            detail: "SAP value contains no digits."
        ]
    }

    List<Option> matches = sapToOptions[sap] ?: []

    if (matches.size() == 1) {
        String value = matches[0].value?.toString()
        Map optInfo = inspectOptionAvailability(issue, value)

        return [
            value: value,
            source: "${typePrefix}_EXISTING_OPTION",
            optionInfo: optInfo,
            detail: "SAP resolved to exactly one existing Project Information option."
        ]
    }

    if (matches.size() > 1) {
        String oldPI = extractDbRow(issue.getCustomFieldValue(cfOldPI))?.trim()
        if (oldPI) {
            Option exact = matches.find { normalize(it.value?.toString()) == normalize(oldPI) }
            if (exact) {
                String exactValue = exact.value?.toString()
                Map optInfo = inspectOptionAvailability(issue, exactValue)

                return [
                    value: exactValue,
                    source: "${typePrefix}_EXISTING_OPTION_DISAMBIGUATED_BY_OLD_PI",
                    optionInfo: optInfo,
                    detail: "SAP matched multiple options but old Project Information disambiguated the value."
                ]
            }
        }
    }

    String label = String.format(LABEL_MULTIPLE, sap)
    Map optInfo = inspectOptionAvailability(issue, label)

    return [
        value: label,
        source: "${typePrefix}_MULTIPLE_REQUIRED",
        optionInfo: optInfo,
        detail: "SAP matched multiple options. Migration would use '${label}'."
    ]
}

String historyLabel = String.format(LABEL_HISTORY, sap)
Map optInfo = inspectOptionAvailability(issue, historyLabel)
return [

```

```

        value: historyLabel,
        source: "${typePrefix}_HISTORY_REQUIRED",
        optionInfo: optInfo,
        detail: "SAP did not match any option. Migration would use '${historyLabel}'."
    ]
}

private Map buildSimulation(
    Issue issue,
    String simulatedValue,
    String simulatedOverrideKey,
    String branch,
    String sourceIssue,
    String authorityValue,
    Map direct,
    String extraDetail
) {
    Map optionInfo = inspectOptionAvailability(issue, simulatedValue)

    String detail = [
        direct?.detail,
        extraDetail
    ].findAll { it?.toString()?.trim() }.join(" ")

    return [
        simulatedPi: simulatedValue ?: "",
        simulatedText: simulatedValue ?: "",
        simulatedOverrideKey: simulatedOverrideKey ?: "",
        branch: branch,
        sourceIssue: sourceIssue ?: "",
        authorityValue: authorityValue ?: "",
        directResolutionSource: direct?.source ?: "UNKNOWN",
        result: "SIMULATED",

        detail: detail,
        optionInfo: optionInfo
    ]
}

private Map buildUnresolvedSimulation(Issue issue, String branch, String detail) {
    return [
        simulatedPi: "",
        simulatedText: "",
        simulatedOverrideKey: "",
        branch: branch,
        sourceIssue: "",
        authorityValue: "",
        directResolutionSource: getDirectResolution(issue)?.source ?: "NO_DIRECT_VALUE",
        result: branch,
        detail: detail ?: "",
        optionInfo: emptyOptionInfo(issue)
    ]
}

private Map unresolvedSynthetic(String issueKey, String result, String detail) {
    return [
        simulatedPi: "",
        simulatedText: "",
        simulatedOverrideKey: "",
        branch: result,
        sourceIssue: "",
        authorityValue: "",
        directResolutionSource: "NO_DIRECT_VALUE",
        result: result,
        detail: "${issueKey}: ${detail}",
        optionInfo: [
            contextId: "",
            existsInContext: false,
            disabledInContext: false,
            existsInAnyContext: false,
            legacyOptionAction: "NONE",
            optionLifecycleAction: "NONE"
        ]
    ]
}

private Map inheritedDirectInfo() {
    return [
        value: "",
        source: "INHERITED",
        oldRaw: "",
        oldExtracted: "",
        oldSap: "",
        sapInput: "",
        sapScripted: "",
        detail: "Issue has no direct migration value and inherits from parent authority."
    ]
}

private String buildCsvLineAndUpdateCounters(Issue issue) {
    Map sim = simulationCache[issue.id] ?: buildUnresolvedSimulation(issue, "NOT_SIMULATED", "Issue was not simulated.")
    Map direct = getDirectResolution(issue)
    Map optionInfo = sim.optionInfo ?: emptyOptionInfo(issue)

    Issue parent = getParent(issue)
    boolean parentInOutputScope = parent ? outputIssuesById.containsKey(parent.id) : false

    String currentPi = getProjectInformationNValue(issue)

```

```

String currentText = getTextMirrorValue(issue)
String currentOverrideKey = getOverrideKey(issue)

String simulatedPi = sim.simulatedPi ?: ""
String simulatedText = sim.simulatedText ?: ""
String simulatedOverrideKey = sim.simulatedOverrideKey ?: ""

boolean piChange = normalize(currentPi) != normalize(simulatedPi)
boolean textChange = normalize(currentText) != normalize(simulatedText)
boolean overrideChange = normalize(currentOverrideKey) != normalize(simulatedOverrideKey)

Map conflict = detectConflict(
    currentPi,
    simulatedPi,
    currentText,
    simulatedText,
    currentOverrideKey,
    simulatedOverrideKey
)

String finalResult = determineFinalResult(
    sim,
    optionInfo,
    piChange,
    textChange,
    overrideChange,
    conflict
)

updateOutputCounters(finalResult, sim, optionInfo, conflict)

List<String> values = [
    issue.key,
    issue.projectObject?.key ?: "",
    issue.issueType?.name ?: "",
    issue.summary ?: "",
    parent?.key ?: "",
    parentInOutputScope.toString(),

    currentPi,
    simulatedPi,
    currentText,
    simulatedText,
    currentOverrideKey,
    simulatedOverrideKey,

    direct.oldRaw ?: "",
    direct.oldExtracted ?: "",
    direct.oldSap ?: "",
    direct.sapInput ?: "",
    direct.sapScripted ?: "",

    sim.directResolutionSource ?: direct.source ?: "",
    optionInfo.contextId?.toString() ?: "",
    optionInfo.existsInContext?.toString() ?: "false",
    optionInfo.disabledInContext?.toString() ?: "false",
    optionInfo.existsInAnyContext?.toString() ?: "false",
    optionInfo.legacyOptionAction ?: "NONE",
    optionInfo.optionLifecycleAction ?: "NONE",

    sim.branch ?: "",
    sim.sourceIssue ?: "",
    sim.authorityValue ?: "",

    piChange.toString(),
    textChange.toString(),
    overrideChange.toString(),

    conflict.conflict?.toString() ?: "false",
    conflict.conflictType ?: "NO_CONFLICT",
    conflict.conflictDetail ?: "",

    finalResult,
    sim.detail ?: ""
]

return values.collect { escapeCsv(it) }.join(SEPARATOR)
}

/**
 * Conflict means:
 *   Current new-field value is already non-empty and differs from simulated migration value.
 *
 * Empty current value is not a conflict. It is simply a would-change.
 *
 * Old child value different from inherited parent value is also not a conflict.
 * That case is handled as OVERRIDE_ROOT according to Variant A.
 */
private Map detectConflict(
    String currentPi,
    String simulatedPi,
    String currentText,
    String simulatedText,
    String currentOverrideKey,
    String simulatedOverrideKey
) {
    boolean piConflict = hasValue(currentPi) && normalize(currentPi) != normalize(simulatedPi)
    boolean textConflict = hasValue(currentText) && normalize(currentText) !=

```

```

        normalize(simulatedText)
        boolean overrideConflict = hasValue(currentOverrideKey) && normalize(currentOverrideKey) !=
            normalize(simulatedOverrideKey)

        boolean conflict = piConflict || textConflict || overrideConflict

        if (!conflict) {
            return [
                conflict: false,
                conflictType: "NO_CONFLICT",
                conflictDetail: ""
            ]
        }

        String type
        if (piConflict && textConflict && overrideConflict) {
            type = "CONFLICT_ALL"
        } else if (piConflict && textConflict) {
            type = "CONFLICT_PI_AND_TEXT"
        } else if (piConflict && overrideConflict) {
            type = "CONFLICT_PI_AND_OVERRIDE"
        } else if (textConflict && overrideConflict) {
            type = "CONFLICT_TEXT_AND_OVERRIDE"
        } else if (piConflict) {
            type = "CONFLICT_PI_VALUE"
        } else if (textConflict) {
            type = "CONFLICT_TEXT_MIRROR"
        } else if (overrideConflict) {
            type = "CONFLICT_OVERRIDE_KEY"
        } else {
            type = "CONFLICT_UNKNOWN"
        }

        List<String> details = []

        if (piConflict) {
            details.add("Current Project Information n '${currentPi}' differs from simulated
                '${simulatedPi}'")
        }

        if (textConflict) {
            details.add("Current text mirror '${currentText}' differs from simulated
                '${simulatedText}'")
        }

        if (overrideConflict) {
            details.add("Current override key '${currentOverrideKey}' differs from simulated
                '${simulatedOverrideKey}'")
        }

        details.add("Manual review required before write migration.")

        return [
            conflict: true,
            conflictType: type,
            conflictDetail: details.join(" ")
        ]
    }

    private String determineFinalResult(
        Map sim,
        Map optionInfo,
        boolean piChange,
        boolean textChange,
        boolean overrideChange,
        Map conflict
    ) {
        String branch = sim.branch?.toString() ?: ""
        String legacyAction = optionInfo?.legacyOptionAction ?: "NONE"

        if (branch == "ERROR" || sim.directResolutionSource == "ERROR") {
            return "ERROR"
        }

        if (branch?.startsWith("UNRESOLVED") || branch == "CYCLE_DETECTED" || branch == "NOT_SIMULATED") {
            return branch
        }

        if (conflict?.conflict == true) {
            if (legacyAction in ["CREATE_LEGACY_OPTION", "CREATE_IN_THIS_CONTEXT_ONLY"]) {
                return "CONFLICT_REQUIRES_LEGACY_OPTION"
            }

            if (legacyAction == "TEMPORARILY_ENABLE_DISABLED_OPTION") {
                return "CONFLICT_REQUIRES_DISABLED_OPTION_HANDLING"
            }

            return "CONFLICT"
        }

        if (legacyAction in ["CREATE_LEGACY_OPTION", "CREATE_IN_THIS_CONTEXT_ONLY"]) {
            return "WOULD_CHANGE_REQUIRES_LEGACY_OPTION"
        }

        if (legacyAction == "TEMPORARILY_ENABLE_DISABLED_OPTION") {
            return "WOULD_CHANGE_REQUIRES_DISABLED_OPTION_HANDLING"
        }
    }

```

```

        if (piChange || textChange || overrideChange) {
            return "WOULD_CHANGE"
        }

        return "OK"
    }

    private void updateOutputCounters(String result, Map sim, Map optionInfo, Map conflict) {
        if (result == "OK") {
            totalOk++
        } else if (result.startsWith("CONFLICT")) {
            totalConflicts++
        } else if (result.startsWith("UNRESOLVED") || result == "CYCLE_DETECTED" || result == "NOT_SIMULATED") {
            totalUnresolved++
        } else if (result == "ERROR") {
            totalErrors++
        } else {
            totalWouldChange++
        }

        String legacyAction = optionInfo?.legacyOptionAction ?: "NONE"
        if (legacyAction in ["CREATE_LEGACY_OPTION", "CREATE_IN_THIS_CONTEXT_ONLY"]) {
            totalLegacyOptionsRequired++
        }

        if (legacyAction == "TEMPORARILY_ENABLE_DISABLED_OPTION") {
            totalDisabledOptionHandling++
        }

        String branch = sim.branch?.toString()
        if (branch == "ROOT_OVERRIDE_OR_STANDALONE_AUTHORITY") {
            totalStandaloneAuthorities++
        } else if (branch == "OVERRIDE_ROOT") {
            totalOverrideRoots++
        } else if (branch == "NESTED_OVERRIDE_ROOT") {
            totalNestedOverrideRoots++
        }
    }

    private Map inspectOptionAvailability(Issue issue, String value) {
        if (!issue || !value || !cfNewPI) {
            return emptyOptionInfo(issue)
        }

        def config = cfNewPI.getRelevantConfig(issue)
        Long configId = config?.id as Long
        String norm = normalize(value)

        Option inContext = configId ? optionsByConfig[configId]?.get(norm) : null
        boolean existsInContext = inContext != null
        boolean disabledInContext = inContext ? isDisabledOption(inContext) : false
        boolean existsInAnyContext = optionOccurrencesByValue.containsKey(norm)

        String legacyAction
        String lifecycleAction

        if (existsInContext && !disabledInContext) {
            legacyAction = "NONE"
            lifecycleAction = "NONE"
        } else if (existsInContext && disabledInContext) {
            legacyAction = "TEMPORARILY_ENABLE_DISABLED_OPTION"
            lifecycleAction = "ENABLE_FOR_MIGRATION_THEN_DISABLE"
        } else if (existsInAnyContext) {
            legacyAction = "CREATE_IN_THIS_CONTEXT_ONLY"
            lifecycleAction = "CREATE_AND_DISABLE_AFTER_MIGRATION"
        } else {
            legacyAction = "CREATE_LEGACY_OPTION"
            lifecycleAction = "CREATE_AND_DISABLE_AFTER_MIGRATION"
        }

        return [
            contextId: configId ?: "",
            existsInContext: existsInContext,
            disabledInContext: disabledInContext,
            existsInAnyContext: existsInAnyContext,
            legacyOptionAction: legacyAction,
            optionLifecycleAction: lifecycleAction
        ]
    }

    private Map emptyOptionInfo(Issue issue) {
        def config = issue && cfNewPI ? cfNewPI.getRelevantConfig(issue) : null

        return [
            contextId: config?.id ?: "",
            existsInContext: false,
            disabledInContext: false,
            existsInAnyContext: false,
            legacyOptionAction: "NONE",
            optionLifecycleAction: "NONE"
        ]
    }

    private Issue getParent(Issue issue) {
        if (!issue) {
            return null
        }
    }

```

```

    if (parentCache.containsKey(issue.id)) {
        return parentCache[issue.id]
    }

    Issue parent = null

    try {
        if (issue.isSubTask()) {
            parent = issue.parentObject
        } else {
            def epic = issue.getCustomFieldValue(cfEpicLink)
            if (epic) {
                parent = resolveIssueFromFieldValue(epic)
            }

            if (!parent) {
                def parentLink = issue.getCustomFieldValue(cfParentLink)
                if (parentLink) {
                    parent = resolveIssueFromFieldValue(parentLink)
                }
            }
        }
    } catch (Exception e) {
        totalErrors++
        log.error("Failed to resolve parent for ${issue.key}: ${e.message}", e)
    }

    parentCache[issue.id] = parent
    return parent
}

private Issue resolveIssueFromFieldValue(Object value) {
    if (!value) {
        return null
    }

    if (value instanceof Issue) {
        return value as Issue
    }

    if (value.hasProperty("key")) {
        try {
            return issueManager.getIssueObject(value.key?.toString())
        } catch (Exception ignored) {
            // Continue with toString fallback.
        }
    }

    String key = value.toString()?.trim()
    if (!key) {
        return null
    }

    try {
        return issueManager.getIssueObject(key)
    } catch (Exception ignored) {
        return null
    }
}

private boolean isInitiative(Issue issue) {
    return issue?.issueType?.name == ISSUE_TYPE_INITIATIVE
}

private String getProjectInformationNValue(Issue issue) {
    def value = issue?.getCustomFieldValue(cfNewPI)

    if (!value) {
        return ""
    }

    if (value instanceof Collection) {
        return value.find { it != null }?.value?.toString() ?: ""
    }

    if (value instanceof Option) {
        return value.value?.toString() ?: ""
    }

    if (value.hasProperty("value")) {
        return value.value?.toString() ?: ""
    }

    return value.toString()
}

private String getTextMirrorValue(Issue issue) {
    return issue?.getCustomFieldValue(cfTextMirror)?.toString() ?: ""
}

private String getOverrideKey(Issue issue) {
    return issue?.getCustomFieldValue(cfOverrideKey)?.toString()?.trim() ?: ""
}

/**
 * Old Project Information in your existing scripts is JSON-like DB row:
 * { rows: [[value]] }
 */

```

```

* If parsing fails and the raw value is not JSON-looking, this method returns
* the raw string as fallback. This makes analyze safer for mixed historical data.
*/
static String extractDbRow(Object raw) {
    if (!raw) {
        return null
    }

    String text = raw.toString()
    if (!text?.trim()) {
        return null
    }

    try {
        def parsed = new JsonSlurper().parseText(text)
        return parsed?.rows?.getAt(0)?.getAt(0)?.toString()
    } catch (Exception ignored) {
        String trimmed = text.trim()

        if (trimmed.startsWith("{") || trimmed.startsWith("[") {
            return null
        }

        return trimmed
    }
}

static String extractSap(String value) {
    if (!value) {
        return null
    }

    def matcher = (value =~ /\[(\d+)\]/)
    return matcher.find() ? matcher.group(1) : null
}

private static String normalize(String value) {
    return value == null ? null : value.trim()
}

private static boolean hasValue(String value) {
    return value != null && value.trim().length() > 0
}

private static boolean isDisabledOption(Option option) {
    if (!option) {
        return false
    }

    try {
        return option.disabled as boolean
    } catch (Exception ignored) {
        try {
            return option.isDisabled()
        } catch (Exception ignored2) {
            return false
        }
    }
}

private File prepareOutputFile() {
    try {
        String jiraHome = jiraHomeComponent?.homePath
        if (!jiraHome) {
            jiraHome = System.getProperty("jira.home", "/tmp")
        }

        File dir = new File(jiraHome, "export/project-info-analyze")
        if (!dir.exists()) {
            dir.mkdirs()
        }

        String timestamp = new SimpleDateFormat("yyyyMMdd_HHmss").format(new Date())
        File file = new File(dir, "project-info-analyze-${timestamp}.csv")
        file.createNewFile()

        return file
    } catch (Exception e) {
        totalErrors++
        log.error("Failed to create output file: ${e.message}", e)
        return null
    }
}

private static String escapeCsv(String value) {
    if (!value) {
        return ""
    }

    if (value.contains(SEPARATOR) || value.contains("\"") || value.contains("\n") ||
        value.contains("\r")) {
        return "\"" + value.replace("\"", "\\\"") + "\""
    }

    return value
}
}

```